Open Name Tags (ONT)

A short, human-readable name — like `alice` — that you truly own. design brief · v0 prototype

Most names online are really *accounts*: a company hands them out, and can rename you, reclaim them, or shut you down. An ONT name is different — it is controlled by a cryptographic key that only you hold. No company is involved, there is nothing to renew or pay rent on, there is no token, and no one (not even ONT's creators) can move or revoke it. Anyone can look up who owns a name and confirm the answer for themselves.

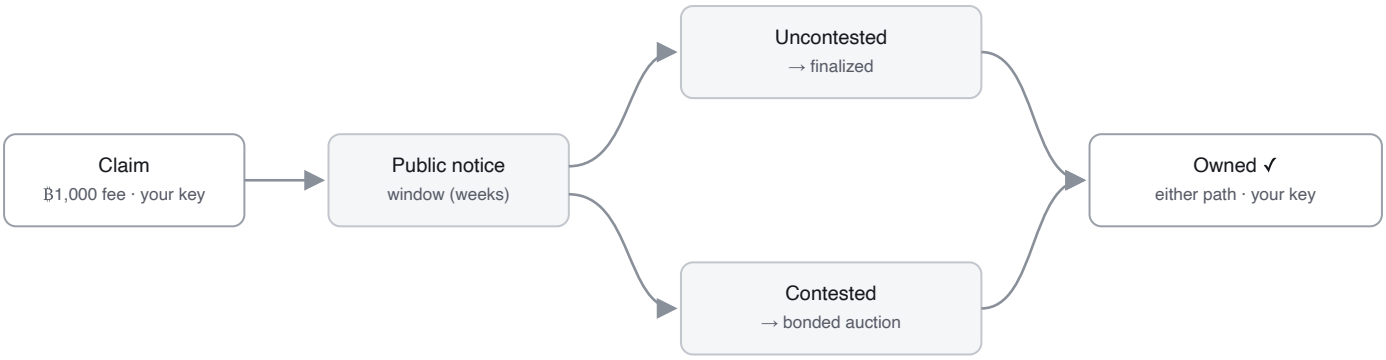
WHAT YOU'D USE ONE FOR

- **A payment handle** — pay `alice` instead of a long address; the name points to wherever she wants to be paid.
- **An identity handle** — one username for open-source / decentralized apps that no platform can reassign or revoke.

HOW YOU GET A NAME

There is one path for every name. It forks only if two people want the same one.

1. **Claim it.** Pay a small, one-time Bitcoin fee — about $1 (**Ƀ1,000**) — paid to Bitcoin's miners, not to ONT.
2. **A public notice window opens** (a few weeks). This is everyone else's chance to contest the name — if someone else wants it too, that sends it to the auction in step 4 instead of finalizing cheaply.
3. **If no one else does, the name is yours.** Thousands of these uncontested claims are bundled into one small Bitcoin record — which is what keeps it cheap across billions of names.
4. **If someone else wants it too, the name is *contested*** and goes to auction: each bidder locks up bitcoin as a **returnable ~one-year bond** (they keep custody; it's released at maturity), and the largest bond wins. Contests can be common early (everyone wants `bitcoin` and dictionary words) but are rare across the long tail at scale, so for most names the auction never happens.



Either path ends in the same place — you own the name, controlled by your key. Most names are never contested (they finalize at window close). If someone else wants the same name, it's contested — settled by an auction, decided by the largest returnable bond; you own and can use it the moment the auction **settles**, and the winning bond just stays posted through maturity (~1 yr) then returns (an ONT rule — break it early and you forfeit the name — not a Bitcoin timelock).

Either way, you end up with the same thing: a globally unique name controlled by your key.

WHAT OWNING A NAME LETS YOU DO

A name is controlled by one key — your **owner key**. With it you can:

- **Point the name somewhere** — at a Bitcoin or Lightning address, a website, and so on — and change it whenever you like. (These mappings live off-chain, signed by your key, so updates are instant, free, and never touch Bitcoin.)
- **Transfer** the name to someone else's key.
- **Set up recovery** ahead of time, so a lost key isn't the end — and only the backup key you chose can use it, so recovery can never become a way for someone to take your name.

HOW IT SCALES — AND THE OPEN PROBLEM

Billions of names can't each be a Bitcoin transaction, so **publishers** batch many claims into one Merkle commitment and anchor only its root — a ~150-byte root that commits to the whole batch *whatever its size*, so the more you batch the lower the per-name cost (**~0.015 vB**/name at ten thousand per batch, less as batches grow). A batch counts only if its miner fee covers the claims inside it, so each name still buys the blockspace it uses.

You pay a publisher for your claim off-chain over Lightning, and it fronts the aggregate miner fee. The honest gap: that payment and the on-chain inclusion aren't yet *atomically* bound — so today you rely on a reputable operator (it verifies payment before anchoring you), and trust-minimizing it likely needs newer Lightning tooling (PTLCs / adapter signatures that release payment only against an inclusion proof). A publisher never holds your *name* — only the batching is intermediated, and you can always claim directly on L1 instead. **v1 starts with a few reputable publishers and minimizes that trust over time.**

WHY YOU CAN TRUST IT

No company, server, or founder decides who owns a name — Bitcoin does. The rules that turn Bitcoin transactions into ownership live in a small, **frozen core — three consensus files** that anyone can audit; a CI test fails if that core grows a dependency beyond Bitcoin/protocol primitives. Run it over Bitcoin's history and you get the same answer everyone else does, and you can check that answer against Bitcoin's own block headers and proof-of-work — so a server that lies about who owns a name gets caught, not believed. The services that help you find and publish names — *resolvers* — only mirror this data; they never decide it.

And no operator is privileged: **anyone can run a resolver or publisher**. Because ownership is fixed by Bitcoin, you don't need a *trusted* node — only a reachable one whose answer you can verify (a lying node is caught; a slow one is routed around). Finding nodes is config-seeded today; a registry-free, on-chain discovery scan is designed, not yet built.

THE NUMBERS WE'RE PROPOSING — SEVERAL ARE PLACEHOLDERS, ALL OPEN TO CHALLENGE

PARAMETER	PROPOSED	STATUS
Claim fee (every name)	**฿1,000** ~$1, sunk, to miners	baseline
Contested-auction min bond	**฿50,000** ~$50, returnable	placeholder
Bond maturity	~52,560 blocks (~1 yr)	test override
Notice window	weeks, height-keyed	placeholder · fairness lever
Data-availability windows	unset	deadline for batch bytes to surface + reorg depth; unpinned
On-chain footprint	~0.015 vB/name; fee = Σ fees	measured

Opening bond for scarce short names.

NAME LENGTH	OPENING BOND	~USD
1 char	**฿100,000,000** (1 BTC)	~$100k
2 char	฿50,000,000	~$50k
3 char	฿25,000,000	~$25k
4 char	฿12,500,000	~$12.5k
5+ char	fee only; ฿50,000 floor if contested	~$1 / ~$50

Only the very short (≤4-char) names carry a high length-scaled opening bond (halving per added character); everything else just uses the flat fee plus a bond if contested.

Least sure of: the **contest rate** is unknown until launch — we assume it's high early (everyone wants `bitcoin`, dictionary words) and low for the long tail (`sallysmith2165`); and the notice window has to be long enough for a competitive early market to form, so premium names aren't swept cheaply before other bidders show up.

STATUS — HONEST (MATURITY, NOT DIRECTION)

Live on a Bitcoin test network (signet), end-to-end: claim, owner-key transfer, owner-signed records, recovery, and a bonded auction bid the resolver accepts — with the consensus code and signatures cross-checked byte-for-byte against a second independent implementation. **Prototype / not yet wired:** the cheap batched-claim path into the live indexer (built and unit-tested, including convergence against a data-withholding adversary); a single-writer publisher; and producers don't yet emit the proofs a phone/browser would check. Not mainnet-ready.

WHAT WE MOST WANT YOU TO PUSH ON

- **Data availability** — we batch claims and anchor only a summary on Bitcoin, leaving the claim data off-chain; our defense if someone withholds it (or a reorg reshuffles it) is a deadline — data that isn't public by a set Bitcoin height simply doesn't count. Is that sound, and should availability be proven on-chain or by timing alone?
- **Publisher trust-minimization** — what's the cleanest Lightning construction (PTLC / adapter signature) to make "pay the publisher" atomic with "claim anchored on-chain," so neither side has to trust the other?
- **Discovery & censorship-resistance** — config-seeded today; is a registry-free, on-chain service-announcement scan the right trustless discovery primitive, with Bitcoin + verification as the only trust root?
- **On-chain footprint** — are the ≤135-byte OP_RETURN events acceptable on mainnet, or is a script/covenant carrier worth a soft-fork dependency?
- **Light-client verification** — a launch blocker, or fine post-launch?
- **Auction form** — open ascending vs. sealed second-price, given MEV and relay-bid timing?
- **Launch fairness** — is a long notice window enough against a day-one rush on premium names, or do we need a decaying launch fee?

Repo: github.com/deekay/ont · the full design brief, prior-art comparison, risk register, and the verification you can run yourself: `docs/ONT_DESIGN_BRIEF.md`.